

LEMBAR KERJA PESERTA DIDIK (LKPD) Algoritma dan Pemrograman Dasar (Python)

BAB 6

Fungsi & Modularisasi Program

Informasi Peserta Didik		
Nama Siswa	:	_____
Kelas / Jurusan	:	_____
No. Absen	:	_____
Tanggal	:	_____

SMK TARUNA JAYA PRAWIRA TUBAN
Program Keahlian: Rekayasa Perangkat Lunak (RPL)

Petunjuk Penggunaan LKPD

Untuk Siswa:

- Pahami konsep fungsi dengan baik sebelum menulis kode — fungsi adalah jantung program yang bersih.
- Perhatikan perbedaan parameter dan argumen, serta return value dan print().
- Uji setiap fungsi yang kamu buat secara terpisah sebelum menggabungkannya.
- Catat hasil percobaan dan output pada kolom yang disediakan.

Untuk Guru:

- LKPD ini dirancang untuk 2 × pertemuan (2 × 45 menit).
- Pertemuan 1: Definisi fungsi, parameter, argumen, return value — Kegiatan 1 & 2.
- Pertemuan 2: Scope variabel, fungsi bawaan Python, fungsi rekursif dasar — Kegiatan 3 & 4.
- Tekankan prinsip DRY (Don't Repeat Yourself) sebagai motivasi utama menggunakan fungsi.

A. Tujuan Pembelajaran

Setelah menyelesaikan kegiatan pada Bab 6 ini, peserta didik diharapkan mampu:

- Menjelaskan konsep fungsi dan manfaat modularisasi program.
- Mendefinisikan fungsi menggunakan kata kunci def dengan sintaks yang benar.
- Membedakan dan menggunakan parameter, argumen, nilai default, dan *args.
- Menggunakan return untuk mengembalikan nilai dari fungsi.
- Memahami konsep scope variabel (lokal vs global).
- Menggunakan fungsi bawaan Python yang umum (len, range, round, abs, dll.).
- Membangun program kalkulator berbasis fungsi yang terstruktur.

Elemen CP	Algoritma dan Pemrograman Dasar
Alokasi Waktu	2 × Pertemuan (2 × 45 menit)
Materi Pokok	def, parameter, return, scope, fungsi bawaan Python
Prasyarat	Bab 3–5 (Operator, Percabangan, Perulangan)
Profil Pelajar Pancasila	Bernalar kritis, Kreatif, Mandiri

B. Materi Ringkas

B.1 Apa itu Fungsi?

Fungsi adalah blok kode yang diberi nama, dapat dipanggil berkali-kali, dan dirancang untuk melakukan tugas tertentu. Fungsi adalah pondasi dari program yang terstruktur, mudah dibaca, dan mudah diperbaiki.

Mengapa kita butuh fungsi?

- Hindari pengulangan kode (prinsip DRY — Don't Repeat Yourself).
- Memecah masalah besar menjadi bagian-bagian kecil yang mudah dikelola.
- Kode lebih mudah dibaca, diuji, dan diperbaiki.
- Fungsi yang sudah dibuat dapat digunakan ulang di program lain (reusable).

Analogi Sehari-hari:

Fungsi seperti resep masakan. Kamu cukup tulis resep satu kali, lalu bisa 'memanggil' resep tersebut kapanpun kamu ingin memasak hidangan yang sama — tanpa harus menulis ulang langkah-langkahnya.

B.2 Mendefinisikan dan Memanggil Fungsi

Fungsi didefinisikan dengan kata kunci `def`, diikuti nama fungsi, tanda kurung, dan titik dua. Blok kode di dalamnya harus diindentasi.

sintaks_fungsi.py

```
# Cara mendefinisikan fungsi
def nama_fungsi(parameter1, parameter2):
    # blok kode fungsi
    # ...
    return hasil    # opsional

# Cara memanggil fungsi
hasil = nama_fungsi(argumen1, argumen2)
```

fungsi_dasar.py

```
# Contoh fungsi tanpa parameter dan tanpa return
def salam_pembuka():
    print("=" * 30)
    print("  Selamat datang!")
    print("=" * 30)
```

```
# Memanggil fungsi – bisa dipanggil berkali-kali!
salam_pembuka()
salam_pembuka()
```

Output:

```
=====
Selamat datang!
=====
Selamat datang!
=====
```

B.3 Parameter dan Argumen

Parameter adalah variabel yang ditentukan saat mendefinisikan fungsi. Argumen adalah nilai nyata yang dikirim saat memanggil fungsi.

Jenis	Penjelasan	Contoh
Parameter biasa	Nilai wajib diberikan saat pemanggilan.	<code>def luas(p, l):</code>
Nilai default	Nilai digunakan jika argumen tidak diberikan.	<code>def salam(nama='teman'):</code>
Keyword argument	Argumen dikirim dengan menyebut nama parameter.	<code>luas(p=5, l=3)</code>
*args	Menerima sejumlah argumen tak tentu sebagai tuple.	<code>def jumlah(*angka):</code>

parameter_argumen.py

```
# Parameter biasa
def luas_persegi_panjang(panjang, lebar):
    print("Luas:", panjang * lebar)

luas_persegi_panjang(8, 5)      # argumen posisional
luas_persegi_panjang(lebar=4, panjang=6)  # keyword argument

# Nilai default
def sapa(nama, salam='Halo'):
    print(salam + ", " + nama + "!")

sapa('Budi')                    # Output: Halo, Budi!
sapa('Ani', 'Selamat pagi')    # Output: Selamat pagi, Ani!
```

B.4 Pernyataan return — Mengembalikan Nilai

Pernyataan return menghentikan eksekusi fungsi dan mengembalikan nilai ke pemanggil. Fungsi tanpa return secara implisit mengembalikan None.

Fungsi dengan print() saja	Fungsi dengan return
<pre>def tambah(a, b): print(a + b) # Tidak bisa disimpan: hasil = tambah(3, 4) # hasil = None !!</pre>	<pre>def tambah(a, b): return a + b # Bisa disimpan & digunakan: hasil = tambah(3, 4) print(hasil * 2) # → 14</pre>
⚠ Nilai tidak bisa digunakan ulang	✓ Nilai dapat disimpan, dihitung, dikombinasi

```
return_value.py

# Fungsi mengembalikan beberapa nilai sekaligus
def statistik(data):
    total = sum(data)
    rata = total / len(data)
    minimum = min(data)
    maksimum = max(data)
    return total, rata, minimum, maksimum

nilai = [75, 82, 90, 68, 95]
tot, avg, mn, mx = statistik(nilai)
print(f"Total: {tot} | Rata: {avg} | Min: {mn} | Max: {mx}")
```

B.5 Scope Variabel — Lokal vs Global

Scope menentukan di mana sebuah variabel bisa diakses. Variabel yang dibuat di dalam fungsi bersifat lokal (hanya bisa diakses di dalam fungsi). Variabel di luar fungsi bersifat global.

```
scope.py

x = 10 # variabel GLOBAL

def fungsi_a():
    x = 99 # variabel LOKAL — berbeda dari x global!
    print("Di dalam fungsi_a, x =", x) # 99

def fungsi_b():
    print("Di dalam fungsi_b, x =", x) # 10 (pakai global)

fungsi_a()
fungsi_b()
print("Di luar fungsi, x =", x) # 10 (tidak berubah)
```

Aturan Praktis Scope:

- Hindari memodifikasi variabel global di dalam fungsi — ini menyebabkan kode sulit dipahami.
- Gunakan parameter dan return untuk bertukar data antara fungsi dan program utama.
- Jika terpaksa butuh global, gunakan kata kunci global (tidak disarankan untuk pemula).

B.6 Fungsi Bawaan Python yang Sering Digunakan

Python memiliki banyak fungsi bawaan (built-in) yang siap pakai tanpa perlu didefinisikan sendiri.

Fungsi	Kegunaan	Contoh
<code>print()</code>	Menampilkan output ke layar	<code>print("Halo")</code> → Halo
<code>input()</code>	Menerima input dari pengguna	<code>input("Nama: ")</code> → str
<code>int()</code> / <code>float()</code> / <code>str()</code>	Konversi tipe data	<code>int('10')</code> → 10
<code>len()</code>	Panjang string/list	<code>len('Python')</code> → 6
<code>range()</code>	Membuat urutan angka	<code>range(1, 6)</code> → 1,2,3,4,5
<code>round()</code>	Membulatkan bilangan	<code>round(3.567, 2)</code> → 3.57
<code>abs()</code>	Nilai mutlak	<code>abs(-15)</code> → 15
<code>max()</code> / <code>min()</code>	Nilai terbesar/terkecil	<code>max(3,7,1)</code> → 7
<code>sum()</code>	Menjumlahkan elemen	<code>sum([1,2,3,4])</code> → 10
<code>type()</code>	Mengecek tipe data	<code>type(3.14)</code> → float

C. Langkah Kerja

Kerjakan setiap kegiatan secara berurutan. Definisikan fungsi sebelum memanggilnya — Python membaca kode dari atas ke bawah!

Kegiatan 1 — Membuat dan Memanggil Fungsi Sederhana

Tujuan: Memahami cara mendefinisikan, memanggil, dan menggunakan return dalam fungsi.

1. Buat file baru bernama fungsi_dasar.py
2. Ketik dan jalankan program berikut:

fungsi_dasar.py

```
# — Fungsi tanpa parameter dan return —
def garis_pemisah():
    print("-" * 35)

# — Fungsi dengan parameter, tanpa return —
def sapa_siswa(nama, kelas):
    print(f"Halo, {nama} dari kelas {kelas}!")

# — Fungsi dengan parameter dan return —
def hitung_luas_segitiga(alas, tinggi):
    luas = 0.5 * alas * tinggi
    return luas

def hitung_keliling_persegi(sisi):
    return 4 * sisi

# — Memanggil semua fungsi —
garis_pemisah()
sapa_siswa("Budi", "X RPL")
sapa_siswa("Ani", "X RPL")
garis_pemisah()

l_segitiga = hitung_luas_segitiga(10, 6)
print(f"Luas segitiga (alas=10, tinggi=6): {l_segitiga}")

k_persegi = hitung_keliling_persegi(7)
print(f"Keliling persegi (sisi=7): {k_persegi}")

# Fungsi langsung dalam ekspresi
print(f"Total luas + keliling = {l_segitiga + k_persegi}")
```

3. Jalankan program. Salin output:

Output program:

4. Jawab pertanyaan berikut:

a) Apa perbedaan yang kamu amati antara fungsi `sapa_siswa()` dan `hitung_luas_segitiga()`?

Jawaban:

b) Mengapa `l_segitiga` + `k_persegi` bisa dihitung langsung, tetapi `sapa_siswa()` tidak bisa seperti itu?

Jawaban:

Kegiatan 2 — Parameter Default dan *args

Tujuan: Menggunakan parameter default dan `*args` untuk membuat fungsi yang fleksibel.

1. Buat file baru bernama `parameter_fleksibel.py`
2. Ketik dan jalankan:

`parameter_fleksibel.py`

```
# Parameter dengan nilai default
def cetak_garis(karakter='-', panjang=30):
    print(karakter * panjang)

cetak_garis()                # pakai default
cetak_garis('-')             # panjang default, karakter '-'
cetak_garis('*', 20)         # kedua argumen diberikan
cetak_garis(panjang=50)     # hanya panjang diganti
```



```
# Fungsi *args – menerima banyak argumen
def hitung_total(*harga):
    print(f"Jumlah item: {len(harga)}")
    print(f"Total      : Rp {sum(harga):,}")
    return sum(harga)

hitung_total(15000, 8000, 25000)
hitung_total(50000, 35000, 12000, 8500, 20000)
```

3. Jalankan dan catat output:

Output program:

4. Modifikasi fungsi `hitung_total()` agar juga menampilkan harga rata-rata. Tulis kode modifikasimu:

Modifikasi `hitung_total()`:

Kegiatan 3 — Memahami Scope Variabel

Tujuan: Membedakan variabel lokal dan global serta memahami dampaknya.

1. Buat file baru bernama `scope_test.py`
2. Sebelum menjalankan, TEBAK dulu output setiap bagian:

scope_test.py

```
# Bagian A
pesan = 'Saya variabel global'

def tampilkan():
    print(pesan)    # pakai variabel global

tampilkan()
print(pesan)

# Bagian B
angka = 100

def ubah_lokal():
    angka = 999    # variabel lokal – BERBEDA dari global!
    print("Di dalam fungsi:", angka)

ubah_lokal()
print("Di luar fungsi:", angka)
```

Tebakanku (sebelum run):	Output sebenarnya:
_____	_____
_____	_____
_____	_____
_____	_____
_____	_____
_____	_____

3. Apakah hasil tebakanmu sesuai? Apa yang kamu pelajari dari percobaan ini?

Jawaban:

Kegiatan 4 — Proyek Mini: Kalkulator Berbasis Fungsi

Tujuan: Mengintegrasikan konsep fungsi, parameter, return, dan percabangan dalam satu program nyata.

Kamu akan membangun kalkulator yang menggunakan fungsi terpisah untuk setiap operasi. Ini adalah contoh nyata modularisasi program!

1. Buat file baru bernama kalkulator.py
2. Lengkapi bagian [???] pada kode berikut:

kalkulator.py — Lengkapi bagian [???]!

```
# =====  
# KALKULATOR BERBASIS FUNGSI  
# SMK TJP Tuban — Kelas X RPL  
# =====  
  
# — Definisi fungsi operasi —  
def tambah(a, b):  
    return [???]  
  
def kurang(a, b):  
    return [???]  
  
def kali(a, b):  
    return [???]  
  
def bagi(a, b):  
    if b == 0:  
        return "Error: Pembagian dengan nol!"  
    return [???]  
  
def pangkat(a, b):  
    return [???]  
  
def modulo(a, b):  
    return [???]  
  
# — Fungsi tampilan —  
def tampilkan_menu():  
    print("\n" + "=" * 35)  
    print("    KALKULATOR PYTHON")  
    print("=" * 35)  
    print("    1. Penjumlahan    (+) ")  
    print("    2. Pengurangan   (-) ")  
    print("    3. Perkalian      (*) ")  
    print("    4. Pembagian     (/) ")  
    print("    5. Perpangkatan  (**) ")  
    print("    6. Modulo        (%) ")  
    print("    0. Keluar")  
    print("=" * 35)  
  
# — Program utama —  
while True:  
    tampilkan_menu()  
    pilihan = input("Pilih operasi (0-6): ")  
  
    if pilihan == '0':  
        print("Terima kasih! Program selesai.")  
        break
```

```
if pilihan in ['1','2','3','4','5','6']:
    a = float(input("Masukkan angka pertama : "))
    b = float(input("Masukkan angka kedua : "))

    if pilihan == '1':    hasil = tambah(a, b)
    elif pilihan == '2': hasil = kurang(a, b)
    elif pilihan == '3': hasil = kali(a, b)
    elif pilihan == '4': hasil = bagi(a, b)
    elif pilihan == '5': hasil = pangkat(a, b)
    else:                 hasil = modulo(a, b)

    print(f"\nHasil = {hasil}")
else:
    print("Pilihan tidak valid!")
```

3. Tulis bagian [??] yang sudah kamu lengkapi:

Baris yang kamu lengkapi:

4. Jalankan program. Uji minimal 4 operasi berbeda. Catat hasilnya:

Output program:

D. Latihan Soal

Kerjakan secara MANDIRI. Untuk soal pemrograman, tulis kode dan tempel outputnya!

Bagian I — Pilihan Ganda

Lingkarkanlah huruf jawaban yang paling tepat!

1. Kata kunci yang digunakan untuk mendefinisikan fungsi di Python adalah ...
 - A. function
 - B. func
 - C. def
 - D. define
2. Apa yang dikembalikan oleh fungsi Python yang tidak memiliki pernyataan return?
 - A. 0
 - B. False
 - C. None
 - D. Error
3. Perhatikan kode:

```
def hitung(x, y=10):  
    return x + y  
print(hitung(5))
```

Output yang dihasilkan adalah ...
 - A. 5
 - B. 10
 - C. 15
 - D. Error
4. Manakah pernyataan yang BENAR tentang variabel lokal?
 - A. Bisa diakses dari mana saja
 - B. Hanya bisa diakses di dalam fungsi tempat ia didefinisikan
 - C. Nilainya tidak berubah
 - D. Sama dengan variabel global

5. Fungsi bawaan Python mana yang digunakan untuk menghitung panjang sebuah string?

- A. count()
- B. size()
- C. length()
- D. len()

6. Apa kegunaan *args dalam definisi fungsi?

- A. Membuat parameter wajib
- B. Menerima sejumlah argumen tak tentu
- C. Mengembalikan banyak nilai
- D. Mendefinisikan nilai default

Bagian II — Analisis dan Prediksi Output

Tentukan output dari setiap kode berikut TANPA menjalankan program!

Soal 1.

```
def kali_dua(x):  
    return x * 2  
  
print(kali_dua(7))  
print(kali_dua(kali_dua(3)))
```

Prediksi output	
Alasan singkat	

Soal 2.

```
def cek_genap(n):  
    if n % 2 == 0:  
        return True  
    return False  
  
for i in range(1, 6):  
    if cek_genap(i):  
        print(i, end=' ')
```

Prediksi output	
Alasan singkat	

Soal 3.

```
def hitung(*angka):  
    return sum(angka), len(angka)  
  
total, banyak = hitung(10, 20, 30, 40)
```

```
print(f'Total={total}, Banyak={banyak}')
```

Prediksi output

Alasan singkat

Bagian III — Soal Pemrograman

Buat fungsi Python untuk setiap soal berikut. Setiap jawaban wajib menggunakan def dan return!

1. Buat fungsi luas_bangun(bentuk, *ukuran) yang menghitung luas berdasarkan parameter:
 - 'persegi': 1 ukuran (sisi) - 'persegi_panjang': 2 ukuran (panjang, lebar) - 'lingkaran': 1 ukuran (jari-jari) - 'segitiga': 2 ukuran (alas, tinggi) Panggil fungsi untuk setiap bentuk dan tampilkan hasilnya.

Kode Program:

Output program:

2. Buat fungsi `konversi_nilai(nilai_angka)` yang mengkonversi nilai angka (0-100) menjadi huruf: A (90-100), B (80-89), C (70-79), D (60-69), E (<60). Buat juga fungsi `cetak_rapor(nama, *daftar_nilai)` yang menerima nama siswa dan beberapa nilai, lalu menampilkan tabel rapor dengan nilai huruf tiap mata pelajaran dan rata-rata.

Code Program:

Output program:

E. Refleksi

Isi bagian refleksi ini setelah menyelesaikan seluruh kegiatan Bab 6!

E.1 Penilaian Diri (Self-Assessment)

Berilah tanda centang (✓) pada kolom yang sesuai!

Pernyataan	Belum	Cukup	Paham	Sangat Paham
Saya memahami konsep fungsi dan manfaatnya dalam modularisasi.				
Saya bisa mendefinisikan fungsi dengan def dan memanggilnya.				
Saya memahami perbedaan parameter, argumen, dan nilai default.				
Saya bisa menggunakan return untuk mengembalikan nilai.				
Saya memahami perbedaan variabel lokal dan global.				

Saya hafal dan bisa menggunakan fungsi bawaan Python yang umum.				
Saya berhasil membangun program kalkulator berbasis fungsi.				

E.2 Pertanyaan Reflektif

1. Jelaskan dengan kata-katamu sendiri apa perbedaan antara fungsi yang hanya menggunakan print() dan fungsi yang menggunakan return. Kapan sebaiknya menggunakan masing-masing?

2. Bayangkan kamu diminta membuat program absensi kelas. Fungsi-fungsi apa saja yang akan kamu buat? Sebutkan minimal 4 fungsi beserta parameter dan return value-nya!

3. Apa manfaat terbesar yang kamu rasakan dari menggunakan fungsi setelah menyelesaikan Bab 6 ini?

E.3 Tantangan Mandiri (Pengayaan)

Buat program Konverter Mata Uang berbasis fungsi!

Spesifikasi program:

- Buat fungsi konversi(jumlah, dari_mata_uang, ke_mata_uang) menggunakan nilai tukar hardcode (IDR, USD, EUR, SGD, JPY).
- Buat fungsi tampilkan_kurs() yang menampilkan semua nilai tukar terhadap IDR dalam format tabel.
- Buat fungsi menu_konversi() sebagai program utama dengan loop while sampai pengguna memilih keluar.
- Gunakan minimal 5 fungsi yang berbeda dalam program.

Tulis kode dan tempel output di sini:

Mengetahui,
Guru Pengampu

Tuban, _____
Peserta Didik,

(_____)

(_____)